software components prebuilt to inspect the input text according to various rules. If the user searches for "K8s", for example, we should be able to fetch books on Kubernetes. Similarly, if search sentences include emojis such as ☕ (coffee), the search engine should be able to extract coffee-appropriate results. These and many more search criteria are honored by the engine due to the way we configure the analyzers.

In this chapter, we examine the mechanics of text extraction and analysis in detail. We begin by looking at one of the common analyzers—the `standard` analyzer—which is a default analyzer that lets us work easily on English text by tokenizing words using whitespace and punctuation as well as lowercasing the final tokens. It also has support for customization: that is, if we want to stop indexing a set of predefined values (maybe common words like *a*, *an*, *the*, *and*, *if*, and so on, or swear words), we can customize the analyzer to do so. In addition to working with the `standard` analyzer, later in the chapter, we look at prebuilt analyzers such as `keyword`, `simple`, `stop`, `whitespace`, `pattern`, and others. We also get our hands dirty with language-specific analyzers for English, German, Spanish, French, Hindi, and so on. We begin with an overview of what text analysis is all about.

> **NOTE** The code for this chapter is available on GitHub (http://mng.bz/JgYQ) and on the book's website (https://www.manning.com/books/elasticsearch-in-action-second-edition).

7.1 Overview

Elasticsearch stores both structured and unstructured data. As we've seen in previous chapters, working with structured data is straightforward. We match documents for the given query and return the result: for example, retrieving a customer's information by their email address, finding canceled flights between dates, getting the sales figures for the last quarter, fetching a list of patients assigned to a surgeon on a given day, and so on. The results are definitive: results are returned if the query matches the documents or not if the query does not match.

Querying unstructured data, on the other hand, involves determining whether a document matches the query and how relevant the document is to the query (how well the document matches). For example, searching for "Konda" across book titles should fetch *Elasticsearch in Action* and other books I've written but not anaconda-related movies.

Analyzers applied during search

Text is analyzed when the data is indexed as well as at querying time. Just as a field is analyzed during indexing, a search query goes through the same process. The same analyzer is often used during search, but our requirements may make a different analyzer more appropriate. Section 7.5 discusses using different analyzers for search and specifying required analyzers for indexing.